

University of Central Punjab

Faculty of Information Technology

Object Oriented Programming

Fall 2025

Lab 11	
Topic	Classes in C++, access specifiers, objects, methods, constructors, destructors, Inheritance, virtual and overridden members, assigning subclass object to superclass pointers, virtual destructors and Abstract class with pure virtual functions.
Objective	Making students familiarize with the concepts of constructors, destructors, Use inheritance, Polymorphism, overriding members and functions, Observing behavior by assigning objects to superclass pointer and Abstract class.
CLO	Apply object-oriented programming principles to implement real-world problems.

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.
- Plan your code carefully on a piece of paper before you implement it.
- Name of the program should be same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
 - o **Variables:** firstName; (no underscores allowed)
 - o **Function:** getName(); (no underscores allowed)
 - o **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 1:

Virtual Functions and Method Overriding

Base Class: Vehicle

Attributes:

- **fuelCapacity (double)** – Capacity of the fuel tank in liters.
- **distanceTravelled (double)** – Distance travelled by the vehicle in kilometers.

Functions:

- a) **Constructor:** Implement a **parameterized constructor** to initialize fuelCapacity and distanceTravelled.
- b) **display()** – Implement a function that prints the fuel capacity and distance travelled.

Virtual Function:

- c) **virtual double calculateFuelEfficiency()** – A virtual function that will be overridden in derived classes. The function will return fuel efficiency calculated as distanceTravelled / fuelCapacity.

Derived Class (From Vehicle): Car

Attributes:

- **maxSpeed (int)** – Maximum speed limit for Car
- **speed (int)** – Current speed for Car

Functions:

- a) **Constructor:** Implement a constructor that calls the base class constructor and initializes additional attributes.

- b) **Override the calculateFuelEfficiency() function:** to account for conditions affecting fuel efficiency. Such as, if car's speed is within maximum allowed speed, then increase the efficiency by 4.

Derived Class (From Vehicle): Truck

Attributes:

- **payloadCapacity (int)** – Pay load capacity for in Tons.

Functions:

- a) **Constructor:** Implement a constructor that calls the base class constructor and initializes additional attributes.
- b) **Override the calculateFuelEfficiency() function** to account for conditions specific to trucks. Such as, decrease efficiency by 7% in case load capacity exceeds than 30 tons.

Main Function:

1. Create an object of the base class Vehicle.
2. Create an object of the derived class Car.
3. Create an object of the derived class Truck.
4. Call the display function for all the objects and observe their outputs.
5. Call the **calculateFuelEfficiency** function for all the objects and observe their outputs.
6. Now assign a subclass object (**Car or Truck**) to a superclass object (**Vehicle**) and try to call the calculateFuelEfficiency function using the superclass object; observe the behavior.
7. Now assign a subclass object to superclass pointer, and try to call **calculateFuelEfficiency** function using the superclass pointer.

Task 2:

Diamond Problem Resolution (Without Virtual Inheritance)

Base Class: Person

Attributes:

- **name (char *)** – Name of the person.
- **age (int)** – Age of the person.

Functions:

- a) **Default Constructor:** Implement a default constructor that prints a message like "Person constructor called."
- b) **display()** – Implement a function that prints the name and age of the person.

Derived Class(from Person): Student

Attributes:

- Inherits from **Person**.

Functions:

- a) **Default Constructor:** Implement a default constructor that prints a message like "Student constructor called."

Derived Class(from Person): Teacher

Attributes:

- Inherits from **Person**.

Functions:

- a) **Default Constructor:** Implement a default constructor that prints a message "Teacher constructor called."

Derived Class(from Teacher & Student): TA

Attributes:

- Inherits from both **Student** and **Teacher**.

Functions:

- a) **Default Constructor:** Implement a default constructor that prints a message like "TA constructor called."

Main Function:

1. Create an object of the **TA class** using the default constructor.
2. Observe the output to see the order, in which constructors are called.
3. Call base class **display()** function and check for ambiguity when calling the **display** function.

Instructions for Students:

1. **Display Function Usage:** Instead of using getter and setter functions, you can call the **display()** function in the base class **Person** to retrieve and display its attributes.
2. **Ambiguity in Function Calls:** When you call **display()** through the **TA** object, you will encounter ambiguity due to multiple inheritance. You should explore how to resolve this by using scope resolution.
3. **Behavior Observations:** Observe the behavior of constructors to understand the order of calls and how attributes are initialized. Use the default constructors to see the messages indicating which constructors are called.
4. **Limitations Discussion:** Discuss the limitations and implications of accessing base class methods when using multiple inheritance and the importance of clear design in class hierarchies.

Task 3;

(Use above task 3 to convert it for virtual Inheritance.)

Diamond Problem Resolution (Using Virtual Inheritance)

Base Class: Person

Attributes:

- name (char *) - Name of the person.
- age (int) - Age of the person.

Functions:

- Constructor:** Implement a parameterized constructor to initialize name and age.
- Default Constructor:** Implement a default constructor that prints a message like "Person constructor called."
- display()** - Implement a function that prints the name and age of the person.

Derived Class(from Person): Student

Using Virtual Inheritance

Attributes:

- **Inherit from Person** using **virtual** inheritance.

Functions:

- Constructor:** Implement a constructor to initialize name and age by calling the Person constructor.
- Default Constructor:** Implement a default constructor that prints a message like "Student constructor called."

Derived Class(from Person): Teacher

Using Virtual Inheritance

Attributes:

- Inherit from **Person** using **virtual** inheritance.

Functions:

- b) **Constructor:** Implement a constructor to initialize name and age by calling the **Person** constructor.
- c) **Default Constructor:** Implement a default constructor that prints a message "Teacher constructor called."

Derived Class(from Teacher & Student): TA

Attributes:

- Inherit from both **Student** and **Teacher**.

Functions:

- b) **Constructor:** Implement a constructor to initialize name, age, and call constructors of **Student** and **Teacher**.
- c) **Default Constructor:** Implement a default constructor that prints a message like "TA constructor called."

Main Function:

1. Create an object of the **TA class** using the default constructor.
2. Observe the output to see the order, in which constructors are called.
3. Call base class **display()** function and check for ambiguity when calling the **display** function.

Instructions for Students:

1. **Virtual Inheritance Usage:** This task uses **virtual inheritance** for both Student and Teacher. This will help resolve ambiguity regarding the Person class when calling the display() function.
2. **Ambiguity Resolution:** When you call **display()** through the TA object, you should find that there is no ambiguity, as the virtual base class will be shared among derived classes.
3. **Behavior Observations:** Observe the behavior of constructors to understand the order of calls and how attributes are initialized. Use the default constructors to see the messages indicating which constructors are called.
4. **Limitations Discussion:** Discuss the limitations and implications of using virtual inheritance, particularly how it can resolve ambiguity problems, and how it affects constructor and destructor behavior.

Task 4:

Overriding and Dynamic Polymorphism

Base Class: Shape

Attributes:

- **color (char *)** - Color of the shape.
- **sides (int)** - Number of sides of the shape.

Functions:

- a) **Parameterized Constructor:** Implement a parameterized constructor to initialize color and sides.
- b) **Virtual Method:** Implement a virtual method **area()** that returns a float representing the area of the shape.
- c) **Virtual Destructor:** Implement a virtual destructor to ensure proper cleanup of derived class objects when deleted through a base class pointer.

Derived Class: Circle

Attributes:

- **radius (float)** – Radius of the circle.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize the radius and call the base class constructor.
- b) **Override Method:** Override the **area()** method to calculate and return the area of the circle.
- c) **Display Method:** Implement a method to display the details of the circle (optional).

Derived Class: Rectangle

Attributes:

- **width (float)** – Width of the rectangle.
- **height (float)** – Height of the rectangle.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize the width, height, and call the base class constructor.
- b) **Override Method:** Override the **area()** method to calculate and return the area of the rectangle.
- c) **Display Method:** Implement a method to display the details of the rectangle (optional).

Main Function:

1. Create an array of **pointers to Shape**.
2. Add multiple objects in the array of Circle and Rectangle class.

3. Loop through the array and call the **area()** function on each element to demonstrate dynamic polymorphism.
4. Display the details of each shape while looping through the array.
5. Verify the outputs against all the elements in the array.

- **Destructors & Virtual Destructor:**

6. Now add **destructors** in each class, along with the **virtual destructor** for **base class Shape**.
7. Each destructor should display a message such as “Base class, Shape destructor”.

Destructor Observation:

8. Now delete any **single object from Circle and Rectangle** subclasses and observe the destructor behavior, noting the order of destructor calls and whether the correct messages are displayed. Delete the objects dynamically at first using the delete keyword.
9. Now loop through the **array and delete the objects in the array** using the **base class pointer** and observe the output.
10. Ensure that the correct destructor messages are displayed, confirming that the virtual destructors are called appropriately for each object.
11. Ensure that the correct derived class destructor is called in all the cases for proper resource cleanup.

Task 5:

Abstract Class and Polymorphism in Banking System

Base Class: BankAccount

Attributes:

- **title (char *)** – Title of the bank account holder.
- **accountNumber (int)** – Unique account number.
- **balance (float)** – Current balance of the account.

Functions:

- a) **Constructor:** Implement a **parameterized constructor** to initialize title, account number, and balance.
- b) **Pure Virtual Functions:**
 - **Deposit(float amount)** – A pure virtual function to deposit an amount into the account.
 - **Withdraw(float amount)** – A pure virtual function to withdraw an amount from the account.
- c) **Virtual Destructor:** Implement a virtual destructor to ensure proper cleanup of derived class objects.

Derived Class: SavingsAccount

Attributes:

- **bonus (int)** – bonus amount, if applicable to the savings account.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize title, account number, balance, and bonus.
- b) **Override Deposit:** Implement the Deposit(float amount) method to increase the balance by the deposited amount. Add bonus as well if the balance exceeds 50,000 each time.
- c) **Override Withdraw:** Implement the Withdraw(float amount) method to decrease the balance by the withdrawn amount, if sufficient funds are available.

Derived Class: CheckingAccount

Attributes:

- **transactionFee (float)** – Fee charged for each transaction.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize title, account number, balance, and transaction fee.
- b) **Override Deposit:** Implement the Deposit(float amount) method to increase the balance by the deposited amount minus the transaction fee.
- c) **Override Withdraw:** Implement the Withdraw(float amount) method to decrease the balance by the withdrawn amount plus the transaction fee, if sufficient funds are available.

Main Function:

1. Create an array of pointers to BankAccount.
2. Add objects of both classes SavingsAccount and CheckingAccount in the array.
3. Simulate transactions for each account type by calling the **Deposit()** and **Withdraw()** methods on each object in the array.
4. Display the final balances of each account after the transactions.